# Patrones de diseño

(Aclaración: se resumen lo que es primer parcial para adelante, ya se supone como sabido temas como principios OO, UML, etc.)

## Patrones de Creación:

- ### Singleton:

  **Porposito**: Clase con una instancia y acceso global para las demás clases.
  **Solución:**
  - La propia clase se instala sola.
  - Permite acceder globalmente a ella, mediante un método.
  - constructor privado así no se la puede instanciar.

  **Problemas:**
  - Se ensucia mucho la clase, es un patrón fácil pero intrusivo.

  Ejemplo: https://refactoring.guru/es/design-patterns/singleton

- ### Multiton (2…N)

  **Propósito:** Singleton pero que se puede instanciar múltiples veces.
  **Solución:** igual que singleton pero se hace un diccionario <ID, instancia> es vez de un atributo, y para obtener la instancia recibis el nombre de ID (nombre instancia).
  **Aplicaciones:**
  - Logs de producción - debug (??).

- ### Factory method

  **Propósito:** Da interfaz para crear objetos pero delega en sus hijas la decisión de qué objeto instanciar.
  **Solución:**
  - Crea una clase abstracta que provee una interfaz para crear objetos .
  - Crea clases factorys que heredan de la abstracta, que implementa EL método definido, creando dicha instancia.
  - Solo se debe usar una vez determinado el objeto a crear.

  **Consecuencias:**
  - Independencia entre clases
  - Rápido y transparente para intercambiar el objeto creado.

  Ejemplo: https://refactoring.guru/es/design-patterns/factory-method

- ### Abstract Factory

  **Propósito:** Da interfaz para crear familia de objetos relacionados, sin decir sus clases concretas..

**Solución:**

- Crea una clase abstracta que provee una interfaz para crear filias de objetos .
- Crea clases factorys que heredan de la factory, que implementan los método definido, creando dichas instancias.
- Solo se debe usar una vez determinado el objeto a crear.

**Consecuencias:**

- Independencia entre clases
- Rápido y transparente para intercambiar el objeto creado.

Ejemplo: https://refactoring.guru/es/design-patterns/abstract-factory

## Patrones de Organización/comportamiento de objetos:

- ### Command

**Propósito:** convierte una solicitud en un objeto independiente que contiene toda la información sobre la solicitud. Parametrizando los métodos con diferentes solicitudes, retrasar o poner en cola la ejecución de una solicitud.

**Solución:**

- Crea clase abstracta/interfaz con un “execute()”.
- Cada hija, implementa ese execute().
- Para usar X execute, se instancia una de las clases y se usa el método.
- Usado para botones y menús en respuesta a lo que haga el usuario ( y en transiciones también).

**Consecuencias:**

- Mantiene referencia a los objetos.
- Órdenes , objetos de primera clase, bastante extensibles.
- Se pueden juntar varias request en una más grande.
- Fácil de añadir ordenes.

Ejemplo: https://refactoring.guru/es/design-patterns/command

- ### Proxy (estructural)

**Propósito:** proporciona sustituto para otro objeto, controlando el acceso al objeto original y hacer algo antes o después de que la request llegue al objeto postal.

**Solución:**

- Crea jerarquía entre objeto posta y proxy. En el proxy, hay referencia al obj original.
- Se redefine toda llamada al proxy integrando código antes de delegarlo al obj posta.
- (Motivación) retrasa el costo de creación-inicialización hasta que se necesita.

**Consecuencias:**

- Agregar funcionalidad, transparentemente, en la aplicación.
- Optimiza, abstrae y establece mecanismos de seguridad en accesos.

Ejemplo: https://refactoring.guru/es/design-patterns/proxy

- ## Facade (estructural)

  **Propósito:** proporciona una interfaz simplificada a cualquier grupo complejo de clases. Minimizando la comunicación y dependencias entre subsistemas.
  **Solución:**
  - Crea Interfaz simple para subsistemas complejos.
  - Queremos dividir en capas nuestros subsistemas.

  **Consecuencias:**
  - Abstrae al cliente con q clases habla, haciendo al subsistema + fácil de usar.
  - Débil acoplamiento entre subsistema y usuario.

  Ejemplo: https://refactoring.guru/es/design-patterns/facade

- ## Strategy

  **Propósito:** Mismo objeto debe poder cambiar su comportamiento en estado de ejecución.
  **Solución:**
  - Delega comportamiento en otros objetos.
  - Arma jerarquía con distintos comportamientos.
  - **Inyecta** el comportamiento al objeto vía constructor o método.

  **Consecuencias:**
  - Elimina condicionales y agrupa comportamiento en familias,
  - No es tan fácil aislar comportamiento, y a veces no es suficiente.

  Ejemplo: https://refactoring.guru/es/design-patterns/strategy

- ## Template (comportamiento)

  ***#Datazo: solo este pide ser una clase abstracta no una interfaz***

  **Propósito:** define el esqueleto de un algoritmo en la superclase pero permite que las subclases sobrescribir pasos del algoritmo sin cambiar su estructura.
  **Solución:**
  - Definir un algoritmo con pasos fijos, pero dejar que las subclases cambien algunos pasos.
  - Evitar duplicar código común entre subclases.
  - Controlar cómo se extienden o modifican esos pasos, manteniendo el flujo general en la clase base.

  **Consecuencias:**
  - Útiles para reutilizar código.
  - Extraen el comportamiento común de las clases en una biblioteca.
  - “Principio Hollywood”: solo clase padre llama a las operaciones de una clase hija.
  - proporcionan el comportamiento predeterminado que puede ser modificado por las subclases

  Ejemplo: https://refactoring.guru/es/design-patterns/template-method

- ## State

  **Propósito:** permite a un objeto alterar su comportamiento cuando su estado interno cambia. Presenta una conexión con 3 estados: *establecida, escuchando y cerrada.*
  **Solución:**
  - Comportamiento de obj depende de su estado, y cambia en tiempo de ejecución según su estado cambia.
  - Las operaciones, tiene muchos condicionales con múltiples ramas que dependen del estado del objeto, siendo su estado varias constantes.

  **Consecuencias:**
  - Transiciones explícitas de estados.
  - Objetos compartibles.
  - Separa el comportamiento según estado, y lo divide en distintas clases de estado.

  Ejemplo: https://refactoring.guru/es/design-patterns/state

- ## Decorator

  **Propósito:** permite añadir funcionalidades a objetos colocando estos objetos dentro de objetos encapsuladores especiales que contienen estas funciones.
  **Solución:**
  - Interfaces de usuario que agregamos comportamiento o propiedades.
  - Se pueden agregar, dinámicamente, objetos individuales.
  - Muy útil cuando la herencia no es viable.

  **Consecuencias:**
  - Mayor flexibilidad que la herencia estática.
  - Evita sobrecarga en clases superiores.
  - Pero tiene la desventaja de que va a tener muchos objetos pequeños.

  Ejemplo: https://refactoring.guru/es/design-patterns/decorator

- ## Composite

  **Propósito:** permite componer objetos en estructuras de árbol y trabajar con esas estructuras como si fueran objetos individuales
  **Solución:**
  - Representar jerarquías de objs parte-todo.
  - Abstrae al usuario de composición d objs y objs individuales, siendo tratados por los usuarios de manera uniforme.

  **Consecuencias:**
  - Permite tratar objetos individuales y compuestos de la misma forma, simplificando el código del cliente y facilitando la extensión.
  - Evita sobrecarga en clases superiores.
  - Pero tiene la desventaja de ser un diseño muy general..

  Ejemplo: https://refactoring.guru/es/design-patterns/composite

- ## Observer

  **Propósito:** define un mecanismo de suscripción para notificar a varios objetos sobre cualquier evento que le suceda al objeto que están observando.
  **Solución:**
  - El objeto con un estado interesante se llama **notificador** (también "sujeto" o "publicador").Los objetos que reaccionan al evento son **notificadores.**
  - El notificador tiene una lista de suscripciones donde añadir o eliminar suscripciones.
  - Ocurre un evento y el notificador **recorre la lista de suscriptores** y llama a un **método de notificación** definido en cada suscriptor.
  - Todos los suscriptores implementan una misma interfaz común. y el notificador se comunica a través de la interfaz.

  Ejemplo: [https://refactoring.guru/es/design-patterns/observer](https://refactoring.guru/es/design-patterns/observer)

# Principios S.O.L.I.D

## S: Single Responsibility Principle (SIP)

Cada clase debe tener **una única razón para cambiar** o **UNA única responsabilidad**

## O: Open-close Principle(OCP)

Clases abiertas a la expansión y cerradas a las modificaciones.Debemos modificar comportamiento sin modificar código, usando herencia o delegación.

## L: liskov substitution principle (LSP)

Si **S** es una subclase de **T** -> objetos de tipo **T** deberían poder ser reemplazados por objetos de tipo **S** sin alterar el comportamiento del programa.
En criollo: Subclase debe poder ser usada en vez de su superclase sin romper nada.
Ejemplo, tenemos ave con método volar, pero luego un pingüino extiende de ave, esto esta mal xq el pingüino no puede volar -> no debe ser una subclase de ave,xq implica q sabe volar las subclases. Es mejor partirlo en una clase animal y otra voladores, y ahí todo bien.

## Interface Segregation principle (ISP)

Ninguna clase debe verse forzada a depender de métodos que no usa. Cada clase, implementa solo lo que necesita.
Es mejor muchas interfaces específicas que 1 sola general.
Ejemplo: si tenes trabajador que trabaja y come, un robot implementa trabajador, puede trabajar pero no comer, ya que en comer, no haría nada (hace return y listo). Entonces, hacete dos interfaces para comer y trabajar y que el robot solo implemente trabajar.

## Dependency Inversion principle (DIP)

Clases de alto nivel, no dependen de las de bajo nivel, deben depender solo de abstracciones, además de que es mucho más barato y fácil modificar las de bajo nivel que las de alto nivel (estas teniendo que cambiarse lo menos posible).

# Paradigma Funcional

## Características:

- inmutabilidad: los datos no cambian una vez creados.
- Funciones puras: la salida depende EXCLUSIVAMENTE de las entradas
- declarativas: se enfoque en el que.
- Funciones de orden superior: permite usar funciones como argumentos o retornados como resultados.
- Transparencia referencial: dada misma entrada, siempre devuelve el mismo resultado.

## Ventajas:

- Legibilidad: código más simple
- Concurrencia segura: evita las *race conditions*
- Escalabilidad: adaptable a sistemas grandes
- Fácil de testear: facil de hacerle unitarias
- Reutilización de código: funciones de alto orden y composición, permiten fácil reutilización.

POO vs Funcional

| Aspecto | Programación Funcional | Programación Orientada a Objetos |
|---|---|---|
| **Principio central** | Transformación de datos con funciones | *Objetos y estado mutable* |
| **Estado** | Inmutable | Mutable |
| **Efectos secundarios** | Evitados | Comunes en los métodos |
| **Modularidad** | Composición de funciones | Clases, Herencia |
| **Concurrencia** | Seguridad natural con datos inmutables | Requiere sincronización |

## Pilares de Funcional

### 1. Funciones Puras

Las funciones siempre retornan con el mismo resultado a la misma entrada. No dependen ni de las entradas, ni variables globales ni variables externas. Y no modifica nada (nada de pasar puntero y que lo modifiquemos)

### 2. Inmutabilidad

La función no cambia el estado de nada, y no puede imprimir ni escribir en archivos, etc.

### 3. Funciones de alto orden

Las funciones son como ciudadanos de primera clase, entonces se podría:

- Pasar funciones como argumentos de otras funciones.
- Valor de retorno de una función a otra función.
- Asignar funciones a variables o almacenarlas en estructura de datos.

### 4. Composición

Funciones que toman otras funciones como argumentos o las devuelven como resultado.

```
const add = (x) => (y) => x + y;
const multiply = (x) => (y) => x * y;

const compose = (f, g) => (x) => f(g(x));
const addAndMultiply = compose(multiply(2), add(3));

console.log(addAndMultiply(4)); // (4 + 3) * 2 = 14
```

## 5. Lazy evaluation( o evaluación diferida)

Expresiones no se evalúan hasta que se las necesita

```
const lazySum = (a, b) => () => a + b;

const result = lazySum(5, 3); // No se evalúa aún
console.log(result()); // 8, se evalúa al invocar
```

## 6. Monedas y efectos secundarios

Estructura que ayuda a manejar efectos secundarios manteniendo el código funcional. (el fetch, catch, then y handell error de Camejo)

## 7. Currying

Transforma una función que recibe múltiples argumentos en una **cadena de funciones** que cada una toma **un solo argumento**.

```
const multiply = (a, b) => a * b;
const curriedMultiply = (a) => (b) => a * b;
const curriedMultiplyBy10 = curriedMultiply(10);

curriedMultiply(5)(10) // Se la invoca 2 veces => 50
curriedMultiplyBy10(5) // 50
curriedMultiplyBy10(3) // 30
```

## 8. Pattern Matching

(No es un principio per se pero es una técnica útil, a priori).
Técnica que permite descomponer estructuras de datos y aplicar condiciones de forma más clara y expresiva.

```
const match = (value, patterns) =>
   (patterns[value] || patterns.default || (() => "No match"))();

const result = match("B", {
   A: () => "Matched A",
   B: () => "Matched B",
   C: () => "Matched C",
   default: () => "Default case",
});

console.log(result); // Output: Matched B
```